Compte Rendu semaine 16/03/2026

Implémenter un code qui permet de retrouver la trajectoire dans le cas où le kart est bloqué

#Dans l'initialisation

```
def __init__(self, env, path_lookahead=3):
    super().__init__(env)
    self.path_lookahead = path_lookahead
    self.agent_positions = []
    self.obs = None
    self.isEnd = False
    self.name = "DemoPilote "

    self.stuck_steps = 0
    self.recovery_steps = 0
    self.en_marche_arriere = False
```

Ajout d'un attribut dans l'initialisation: self.en_marche_arriere = False : C'est un booléen. Il indique au programme si le kart est actuellement en train d'exécuter la stratégie d'évitement ou s'il conduit normalement.

```
def gerer_recul(self, obs, vitesse):
    """ Gère la détection de blocage et la marche arrière """
    phase = obs.get("phase", 0)

    if vitesse < 0.2 and phase > 2: #si vitesse nulle après le départ
        self.stuck_steps += 1
    else:
        self.stuck_steps = 0

    if self.stuck_steps > 5 and not self.en_marche_arriere: #temps de decision d'activer marche arriere
        self.en_marche_arriere = True
        self.recovery_steps = 15 #durée de la marche arriere

    if self.en_marche_arriere:   #execution de marche arriere
        self.recovery_steps -= 1
        if self.recovery_steps <= 0:
            self.en_marche_arriere = False

        #braquage
        correction = self.correction_centrePiste(obs)
        braquage_arriere = 0.85 if correction > 0 else -0.85 #braquage

        return {
            "acceleration": 0.0,
            "steer": braquage_arriere,
            "brake": True, #declencher la marche arriere
            "drift": False,
            "nitro": False,
            "rescue": False,
            "fire": False,
        }
    return None
```

Ajout d'une fonction gerer_recul(self, obs, vitesse).

#Algorithme suivi:

- Vérification de l'état du kart:

- Si le kart est à l'arrêt et que la course est commencée => compter le temps bloqué
- Sinon => réinitialiser le compteur à zéro.

- Prise de décision :

- Si le kart est resté bloqué trop longtemps => déclencher la marche arrière

- Action de secours :

- Si la marche arrière est activé :

  -braquer les roues pour s'éloigner du mur

  -activer marche arrière

  -décompter le temps restant jusqu'à la fin de la manœuvre

  -si temps écoulé => désactiver la marche arrière

#Explications

#Détection:

```
if vitesse < 0.2 and phase > 2:
    self.stuck_steps += 1
else:
    self.stuck_steps = 0
```

Si le kart est lent après le départ => on augmente le compteur de blocage (=incapable d'avancer). Sinon, on le remet à 0 (=kart libre).

#Décision:

```
if self.stuck_steps > 5 and not self.en_marche_arriere:
    self.en marche arrière = True
    self.recovery_steps = 15
```

Si on dépasse un certain seuil ça veut dire qu'on est vraiment bloqué => activation de la marche arrière + détermination de la durée de la marche arrière.

#Exécution:

```
if self.en marche arrière:
    self.recovery_steps -= 1
    if self.recovery_steps <= 0:
        self.en_marche_arriere = False
```

Si on est en marche arrière On diminue le temps restant et si le temps de la marche arrière est écoulé on désactive la marche arrière puis:

```
correction = self.correction_centrePiste(obs)
braquage_arriere = 0.85 if correction > 0 else -0.85
return {
        "acceleration": 0.0,
        "steer": braquage_arriere,
        "brake": True,
        "drift": False,
        "nitro": False,
        "rescue": False,
         "fire": False, }
```

On braque pour que le kart se replace dans la bonne direction et ne plus être face au mur ou à l'obstacle et on renvoie le dictionnaire avec les actions à réaliser contenant les nouvelles valeurs.

```
return None
```

Sinon on ne fait rien et donc on ne renvoie rien.

#Dans choose_action

```
action_secours = self.gerer_recul(obs, speed)
if action_secours is not None:
    return action_secours
```

Si action_secours a une valeur (= pas None) => on renvoie l'ordre de reculer et le reste du code (pilotage normal) n'est même pas lu.

[Réfléchir à une implémentation d'un code sur le path_lookahead pour permettre au kart d'être plus dynamique et lui permettre d'anticiper plus rapidement]

Actuellement notre path_lookahead est une valeur fixe. Pour rendre le kart plus dynamique, il faut rendre le path_lookahead adaptatif => il doit regarder loin en ligne droite (= pour prendre de la vitesse) et regarder plus près en virage serré (= pour être précis).

path_lookahead doit être dynamique car si le kart regarde toujours au même endroit:

-en ligne droite => manque d'anticipation + fait des corrections inutiles.

-en virage serré : Il coupe sa trajectoire trop tôt ou rate l'entrée car le point qu'il vise est trop éloigné et donc l'emmène vers l'extérieur de la piste.

Stratégie d'implémentation: au lieu d'avoir un self.path_lookahead constant, on peut calculer une valeur dynamique dans choose_action qui se base sur la courbure de la piste.

-si la courbure est élevée (virage) lookahead diminue (on regarde près).

-si la courbure est faible (ligne droite) lookahead augmente (on anticipe loin).

Idée de code:

```
def get_dynamic_lookahead(self, obs):
    #on utilise une valeur de base et on l'ajuste
    base_lookahead = 5

    #récupère la courbure
    virages = self.detectVirage(obs)
    if len(virages) > 0:
        curvature = abs(virages[0]["curvature"])
        #si le virage est très serré => augmenter le lookahead
        if curvature > 0.5:
            return 4
        elif curvature > 0.2:
            return 2

    #ligne droite ou virage léger => regarder plus loin
        return 8
```

Optimisation automatique des parametres avec optuna

-L'objectif était de mettre en place un système d'**optimisation automatique des paramètres de conduite** du kart (le fichier configDEmoPilote.yaml) en utilisant la bibliothèque **Optuna**.

Développement du script optimise_kart.py :

1. Proposer des valeurs de conduite (correction, courbure) de manière intelligente.
2. Modifier automatiquement le fichier de configuration YAML.
3. Lancer des sessions de course en arrière-plan via subprocess.
4. Analyser le terminal pour extraire le temps de course et l'optimiser.

2. La Fonction Objectif :

À chaque essai (*Trial*), la fonction `objective` exécute quatre étapes clés :

- **Source des paramètres :** Via suggest_float, Optuna propose des valeurs pour correction et curvature en se basant sur les résultats des essais précédents.
- **Injection de configuration :** Le script ouvre le fichier configDEmoPilote.yaml, écrase les anciennes valeurs par les nouvelles, et sauvegarde le fichier. Cela permet à l'agent d'utiliser les paramètres d'Optuna sans intervention manuelle.
- **Exécution du simulateur :** La commande subprocess.run lance le jeu. L'utilisation de cwd = “src/main” garantit que le script de course trouve ses dépendances. On utilise captur.output = true pour récupérer tout ce qui s'affiche dans le terminal.
- **Analyse des résultats :** Le script parcourt le texte de sortie ligne par ligne. Lorsqu'il détecte le mot-clé “Time”, il extrait la valeur numérique associée.

Nouvelle implementation pour la fonction detecteVirage():

Problème: les valeurs retournées dans la liste de curvatures dans le précédent code étaient erronées (ex: valeurs jamais négatives, ne suivaient pas les courbes observées durant la course).

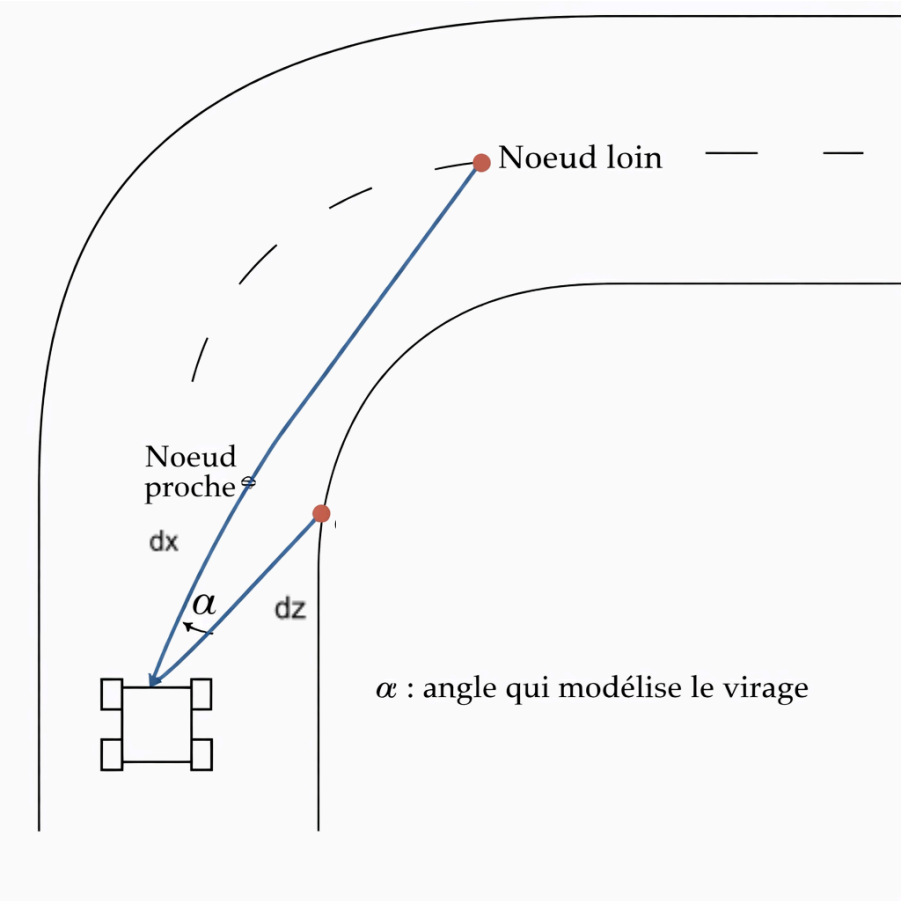
```
    noeuds_piste= obs["paths_start"]
    path_lookahead = 5
    noeud_cour= noeuds_piste[0]
    noeud_loin= noeuds_piste[path_lookahead]

    x1, z1 = noeud_cour[0], noeud_cour[2]
    x2, z2 = noeud_loin[0], noeud_loin[2]

    dx = x2 - x1
    dz = z2 - z1

    angle= np.arctan2(dx,dz)
    return angle
```

Schéma explicatif de la stratégie adoptée pour cette implémentation:



Documentation du code avec Doxygen

Pour faciliter la lecture et la maintenance du code, nous avons mis en place une documentation automatique avec **Doxygen**.

Annotation des fichiers

Tous les fichiers de l'agent ont été commentés en suivant la syntaxe Doxygen pour Python. Chaque fichier, classe, méthode et attribut est documenté avec les balises suivantes :

- `@file` / `@brief` : description du fichier et de chaque élément
- `@param` / `@return` : paramètres et valeurs de retour de chaque méthode
- `@var` : description de chaque attribut d'instance
- `@see` : renvois entre les modules liés

Les fichiers documentés sont : `agent2.py`, `anticipe_kart.py`, `acceleration_kart.py`, `kart_rescue.py`, `react_items.py`, `rival_attack.py`, `steering_piste.py`, `Kart_drift.py`, `optimise_kart.py`.

Génération de la documentation

Le Doxyfile a été créé directement dans le dossier `team2` :